

1. Code Context (Big Picture)

Imagine you have a bunch of **school PDFs** — like your science chapters, history notes, or math formulas — and you wish you had a **study buddy** who could:

- Read all of them
- Remember the important parts
- Answer any questions you ask about them
- Summarize them for revision
- Even make quizzes to test you

This code builds exactly that kind of **AI-powered study buddy**.

You upload your PDFs (or text files), it:

1. Reads the text
2. Breaks it into small readable chunks
3. Turns each chunk into a “memory” (numbers that represent the meaning — called embeddings)
4. Stores all these “memories” in a **search brain** (FAISS index)
5. When you ask a question, it finds the most relevant memories and asks the AI model to answer using them

2. Line-by-Line (Grouped) Breakdown

Imports — Bringing in Tools

```
python
CopyEdit
import streamlit as st
import pdfplumber
import os
import json
import numpy as np
import faiss
from groq import Groq
from sentence_transformers import SentenceTransformer
from dataclasses import dataclass
```

from typing import List, Dict

- **What's happening?**

We're collecting different tools (libraries) so our code can do things like:

- Make a webpage (streamlit)
- Read PDFs (pdfplumber)
- Handle files (os)
- Work with numbers (numpy)
- Store and search memories (faiss)
- Talk to AI (groq)
- Turn text into number meanings (SentenceTransformer)
- Organize data neatly (dataclass, typing)

 **Analogy:** Like packing your school bag with math book, ruler, calculator, and notebook — each has a purpose.

Configuration — Setting Up

python

CopyEdit

```
MODEL_NAME = "llama-3.1-8b-instant"
```

```
EMBED_MODEL_NAME = "all-MiniLM-L6-v2"
```

```
GROQ_API_KEY = "YOUR_API_KEY_HERE"
```

- We pick:
 - Which AI model to talk to
 - Which embedding model to use
 - The API key (password) to talk to Groq's AI service
-

Dataclass — The Memory Box

python

CopyEdit

```
@dataclass
```

```
class DocChunk:
```

```
    id: str
```

```
    text: str
```

meta: Dict

embedding: np.ndarray

- Think of this as a **labelled memory box** that stores:
 - id: name for the chunk
 - text: the actual piece of your notes
 - meta: extra info like filename
 - embedding: the vector (AI's memory form of that chunk)
-

Reading PDFs

python

CopyEdit

```
def extract_text_from_pdf_file(uploaded_file):
```

```
    text = ""
```

```
    with pdfplumber.open(uploaded_file) as pdf:
```

```
        for page in pdf.pages:
```

```
            text += page.extract_text() + "\n"
```

```
    return text
```

- Opens your PDF, goes page by page, and grabs all the text into one string.
 -  **Analogy:** Like flipping through your notebook and copying all words into a single sheet.
-

Chunking Text

python

CopyEdit

```
def chunk_text(text, chunk_size=400, overlap=50):
```

```
    ...
```

- Breaks the big text into smaller overlapping pieces so the AI can process them better.
 - Overlap keeps some context so the AI doesn't lose meaning between chunks.
 -  **Analogy:** Like splitting your notes into flashcards, with a little repetition between cards.
-

Loading and Using Embeddings

python

CopyEdit

@st.cache_resource

```
def load_embed_model():
```

```
    return SentenceTransformer(EMBED_MODEL_NAME)
```

```
def embed_texts(model, texts):
```

```
    embeddings = model.encode(texts, convert_to_numpy=True)
```

```
    faiss.normalize_L2(embeddings)
```

```
    return embeddings
```

- Loads the embedding model once (so it's fast next time).
 - Turns each chunk of text into a vector (list of numbers).
 - 📖 Analogy: Like putting each flashcard in a position on a huge "meaning map."
-

FAISS Index

python

CopyEdit

```
def build_faiss_index(embeddings):
```

```
    index = faiss.IndexFlatIP(embeddings.shape[1])
```

```
    index.add(embeddings)
```

```
    return index
```

- Creates a special "search brain" that can quickly find which stored vectors are closest to your question vector.
 - 📖 Analogy: Like a super-organized library card catalog.
-

Asking the AI

python

CopyEdit

```
def call_groq_chat(system, user, temperature=0.3, max_tokens=1024):
```

```
    client = Groq(api_key=GROQ_API_KEY)
```

```
    chat_completion = client.chat.completions.create(
```

```
        model=MODEL_NAME,
```

```

messages=[
    {"role": "system", "content": system},
    {"role": "user", "content": user}
],
temperature=temperature,
max_tokens=max_tokens
)
return chat_completion.choices[0].message.content

```

- Sends the “system” role (instructions on how to behave) and the “user” question to Groq.
- Groq sends back an answer from the AI.

Streamlit App Layout

python

CopyEdit

st.set_page_config(...)

st.title(...)

- Sets up the webpage title and layout.
- Adds sidebar widgets for uploading files and setting chunk sizes.
- Adds tabs for:
 - Asking questions
 - Summarizing
 - Making quizzes
 - Giving study tips

3. Library Functions (in simple words)

Library	Purpose
streamlit	Makes it easy to build interactive web apps in Python
pdfplumber	Reads text from PDF pages
numpy	Handles lists of numbers (arrays) quickly
faiss	Searches for similar vectors very fast

Library	Purpose
groq	Lets us talk to Groq's AI models
sentence-transformers	Turns text into embeddings (vector form)

4. Related Concepts

- **Variable:** A box with a label where we store data (like MODEL_NAME).
 - **Function:** A mini-program that does one job (like extract_text_from_pdf_file).
 - **Loop:** Repeats actions (like for page in pdf.pages).
 - **List/Array:** A collection of items (like multiple embeddings).
 - **Dictionary (dict):** Stores data in key: value pairs, like a phone book.
-

5. Visual Aids to Help

1. **Flow Diagram:**
Upload PDF → Extract text → Chunk text → Make embeddings → Store in FAISS → Ask question → Retrieve chunks → Send to AI → Show answer.
 2. **Chunk Example:** Show how one page is split into overlapping text pieces.
 3. **Embedding Map:** A 2D plot showing similar chunks close together.
 4. **Sidebar + Main Area Screenshot:** To connect code to what the user sees.
-

6. Summary

- This code makes a **personal AI tutor** for your PDFs.
- It:
 1. Reads your notes
 2. Splits them into smaller pieces
 3. Stores them in a special memory format
 4. Finds relevant pieces when you ask questions
 5. Uses AI to answer, summarize, and quiz you
- You learned about **functions, variables, loops, lists, dictionaries**, and some powerful AI tools.
- The main takeaway: AI can't just "read" your notes — it needs them broken into **embeddings** and **indexed** so it can quickly find and use them.